SCORE90X

FIFA WORLD CUP 2026 — LIVE SCORE PLATFORM

Project Documentation

Repository

github.com/Abu10thahir7-github/WorldCup-Score90X

Author: Abu Thahir

Stack: MERN · Next.js 16 · TypeScript · Express

Document Date: July 2026

1. Project Overview

Score90X is a full-stack, production-oriented sports dashboard built for the FIFA World Cup 2026. It delivers live match scores, standings, team and player profiles, a knockout-stage bracket, and top-scorer leaderboards through a cinematic, dark-themed interface. The project is split into two independently deployable applications: a Next.js frontend and an Express/TypeScript backend that proxies and caches data from an external football data API.

1.1 Goals

- Provide real-time (near-live) match scores and tournament standings for the 2026 World Cup.
- Offer rich team and player profile pages with a consistent, premium visual identity.
- Visualize the knockout stage as an interactive bracket.
- Keep external API usage efficient through server-side and client-side caching layers.
- Demonstrate a scalable MERN/Next.js architecture suitable for a freelance portfolio piece.

1.2 Repository

The project is hosted as a monorepo-style repository with two top-level applications:

- backend/ — Node.js + TypeScript + Express API server
- frontend/ — Next.js 16 (App Router) + TypeScript client application

2. Technology Stack

Layer	Technology	Purpose
Frontend Framework	Next.js 16 (App Router)	React 19 UI with file-system routing
Language	TypeScript	Type safety across frontend and backend
Styling	Tailwind CSS v4	Utility-first dark theme styling
Animation	Framer Motion	Page/section transitions and micro-interactions
Data Fetching (client)	TanStack Query (React Query)	Caching, retries, background refetch control
State Management	Zustand	Lightweight UI/selection state (selected match/team, menu state)
HTTP Client	Axios + axios-cache-interceptor	API communication with response caching
Forms & Validation	React Hook Form + Zod	Typed form handling (scaffolded)
Charts	Recharts	Data visualization support
Icons	lucide-react	Icon system used across cards and UI
Backend Framework	Express 5	REST API server

Layer	Technology	Purpose
Backend Language	TypeScript (ts-node-dev)	Typed backend development
HTTP Client (server)	Axios	Calls to the external football-data API
Security	Helmet, CORS, express-rate-limit	HTTP hardening and abuse protection
Compression	compression	Gzip response compression
Scheduling	node-cron	Reserved for scheduled data refresh jobs
Database (scaffolded)	MongoDB via Mongoose	Present in dependencies; not yet wired into the running server
Validation	Zod	Schema validation (shared dependency across both apps)
Package Manager	pnpm	Used for both frontend and backend

3. System Architecture

Score90X follows a classic client/server split. The Next.js frontend never talks to the external football-data provider directly — all outbound requests are routed through the Express backend, which centralizes API-key handling, rate limiting, and in-memory response caching before returning normalized JSON to the client.

3.1 High-Level Data Flow

```
Browser (Next.js Client / Server Components)
        │
        │  Axios (axiosClient) + axios-cache-interceptor
        ▼
Express Backend  (/api/worldcup/*)
        │
        │  In-memory TTL cache (utils/cache.ts)
        ▼
football-data.org REST API  (X-Auth-Token secured)
```

3.2 Caching Strategy

Caching happens at two layers. On the backend, getCachedData() wraps every repository call with an in-memory Map keyed by resource (e.g. wc-matches, team-{id}), each with its own time-to-live: 5 minutes for live match data, 10 minutes for standings, and 1 hour for relatively static resources like teams and players. On the frontend, axios-cache-interceptor and a lightweight lib/cache.ts wrapper avoid redundant network calls, while TanStack Query manages component-level cache/staleness (30-minute staleTime, 24-hour garbage-collection time).

3.3 Deployment Model

- Frontend: deployable as a standalone Next.js application (e.g. Vercel).
- Backend: deployable as a standalone Node service (e.g. Render/Railway/Fly.io), exposing REST endpoints under /api/worldcup.
- Communication: configured via NEXT_PUBLIC_API_BASE_URL on the frontend pointing to the backend's public URL.

4. Backend Documentation

4.1 Folder Structure

```
backend/
├── src/
│   ├── server.ts                          # App bootstrap: middleware, routes, listener
│   ├── database/
│   │   └── mongodb.ts                     # Reserved Mongoose connection (not yet active)
│   ├── integrations/
│   │   └── footbal-data/
│   │       └── football.client.ts         # Axios client for football-data.org
│   ├── modules/
│   │   ├── worldcup.routes.ts             # Route → controller bindings
│   │   ├── worldcup.controller.ts         # Request/response handling
│   │   ├── worldcup.service.ts            # Caching + orchestration layer
│   │   ├── worldcup.repository.ts         # External API calls
│   │   └── worldcup.mapper.ts             # Normalizes raw match payloads
│   └── utils/
│       └── cache.ts                       # Generic in-memory TTL cache helper
├── package.json
└── tsconfig.json
```

4.2 Request Lifecycle

Each request passes through a layered architecture:

- Routes (worldcup.routes.ts) — maps HTTP verbs/paths to controller functions.
- Controllers (worldcup.controller.ts) — extracts params, calls the service layer, and shapes the { success, data } / { success, message } JSON envelope, catching and logging errors as 500 responses.
- Services (worldcup.service.ts) — wraps each repository call in getCachedData() with a resource-specific TTL.
- Repository (worldcup.repository.ts) — performs the actual Axios call against the football-data.org API.
- Mapper (worldcup.mapper.ts) — normalizes raw match objects into a clean shape (id, status, utcDate, stage, group, matchday, teams, score) before they reach the client.

4.3 API Endpoints

Method	Endpoint	Description	Notes
GET	/	Health check	Plain text "Score90X Backend Running" message
GET	/api/worldcup/matches	All World Cup matches	Cached 5 min; mapped via mapMatch()
GET	/api/worldcup/standings	Group/tournament standings	Cached 10 min
GET	/api/worldcup/teams	All participating teams	Cached 1 hour
GET	/api/worldcup/teams/:id	Single team profile	Cached 1 hour, keyed by team id
GET	/api/worldcup/persons/:id	Player/person profile	Cached 1 hour, keyed by person id

Method	Endpoint	Description	Notes
GET	/api/worldcup/matchesDetails/:id	Single match detail	Cached 5 min, keyed by match id
GET	/api/worldcup/scorers	Top scorer leaderboard	Cached 5 min

4.4 Response Envelope

All endpoints return a consistent JSON shape:

```
// Success
{ "success": true, "data": [...] }

// Failure
{ "success": false, "message": "Failed to fetch matches" }
```

4.5 Middleware & Security

- helmet() — sets secure HTTP response headers.
- compression() — gzip-compresses JSON responses.
- cors() — allows cross-origin requests from the frontend.
- express-rate-limit — 200 requests per IP per 15-minute window; returns a JSON 429 message when exceeded.
- dotenv — loads PORT, FOOTBALL_BASE_URL, and FOOTBALL_API_KEY from a .env file.

4.6 Environment Variables

```
PORT=5000
FOOTBALL_BASE_URL=https://api.football-data.org/v2
FOOTBALL_API_KEY=your_api_token_here
```

4.7 Running the Backend

```
cd backend
pnpm install
pnpm dev        # development (ts-node-dev, auto-restart)
pnpm build       # compiles TypeScript to dist/
pnpm start       # runs compiled dist/server.js
```

4.8 Notes & Known Gaps

- database/mongodb.ts is present as a scaffold but not currently connected in server.ts — MongoDB/Mongoose is a listed dependency but not yet wired into a running data layer.
- node-cron is included as a dependency, presumably intended for scheduled cache warm-ups or background refresh jobs, but no cron job is currently registered in the codebase.
- No formal API versioning or OpenAPI/Swagger spec is present; this document serves as the primary API reference.

5. Frontend Documentation

5.1 Folder Structure

```
frontend/
├── src/
│   ├── app/                        # Next.js App Router — pages & layouts
│   │   ├── layout.tsx              # Root layout: fonts, providers, shell (sidebar/nav/footer)
│   │   ├── page.tsx                # Home page
│   │   ├── overView/page.tsx       # Tournament overview dashboard
│   │   ├── live-matches/page.tsx   # Live matches dashboard
│   │   ├── matches/page.tsx        # Matches list
│   │   ├── matches/[id]/page.tsx   # Match detail page
│   │   ├── teams/page.tsx          # Teams grid
│   │   ├── teams/[id]/page.tsx     # Single team profile page
│   │   ├── players/[id]/page.tsx   # Player profile page
│   │   ├── table-standing/page.tsx  # Standings table
│   │   ├── group-standings/page.tsx  # Group-stage standings
│   │   ├── bracket/page.tsx        # Knockout-stage bracket view
│   │   ├── top-scorers/page.tsx    # Top scorer leaderboard
│   │   ├── search/page.tsx         # Search results page
│   │   ├── loading.tsx / not-found.tsx
│   │   └── global.css
│   ├── components/
│   │   ├── layout/                 # navbar, mobile-navbar, sidenavbar, footer
│   │   ├── OverView.tsx/           # Overview dashboard widgets (banners, mini overviews)
│   │   ├── matches/                # match-card, live-match-card, matchDetails/*
│   │   ├── teams/                  # Team.tsx, TeamCard.tsx, banner, SingleTeam/*
│   │   ├── players/                # PlayerProfileCard, PlayerInformation, PlayerCurrectTeam
│   │   ├── standings/              # tableStanding, groupStanding-table, StandingColumnInfo
│   │   ├── topScores/              # TopScoresCard.tsx
│   │   ├── bracket/                # Bracket.tsx, BracketColumn, MatchCard, Connector,
FinalCard
│   │   ├── search/                 # SearchResults.tsx
│   │   ├── shared/                 # error-message, loading-skeleton, section-title
│   │   └── ui/                     # badge, button, card, input, BackButton, CountDown, etc.
│   ├── features/                   # Feature-area barrel exports (matches, players, standings,
teams, worldcup)
│   ├── hooks/                      # useMatches, useLiveMatches, useTeams, useStandings,
useTopScorers, useBracket, etc.
│   ├── services/                   # worldcup.ts (core API layer) + per-domain service
wrappers
│   ├── store/                      # Zustand stores: matches-store.ts, ui-store.ts
│   ├── providers/                  # query-provider.tsx (TanStack Query), theme-provider.tsx
│   ├── lib/                        # axios.ts, cache.ts, utils.ts, wikipedia-news.ts
│   ├── types/                      # Match, Team, StandingGroup, TopScorer, Person, bracket,
matchDetails
│   ├── constants/routes.ts          # Centralized route path constants
│   └── utils/                       # bracketTree.ts, groupBracket.ts
├── package.json
└── tailwind.config.ts
```

5.2 Application Routes (App Router)

Route	Page	Description
/	Home	Landing/dashboard entry point

Route	Page	Description
/overView	Tournament Overview	Live banner, mini group overview, news, standing guide
/live-matches	Live Matches	Tab-navigated live match dashboard with status indicators
/matches	Matches	Full match list
/matches/[id]	Match Details	Score summary, venue, referees, home/away detail
/teams	Teams	Grid of all participating teams
/teams/[id]	Team Details	Cinematic profile: banner, coach, colors, squad list
/players/[id]	Player Profile	Two-column cinematic player profile
/table-standing	Table Standings	Full tournament standings table
/group-standings	Group Standings	Group-stage standings
/bracket	Knockout Bracket	Interactive bracket tree of the knockout stage
/top-scorers	Top Scorers	Ranked leaderboard of tournament top scorers
/search	Search	Cross-entity search results

5.3 Root Layout & Providers

layout.tsx defines the global shell: Poppins and Inter are loaded via Google Fonts, the page is forced into dark mode by default, and the app is wrapped in QueryProvider (TanStack Query) and ThemeProvider. The visible shell composes a desktop sidebar (SideNavBar), a top Navbar, a scrollable content area for the active route, and a persistent Footer.

5.4 State Management

- TanStack Query — server-state cache for all API-backed data (matches, teams, standings, scorers), configured with a 30-minute staleTime and 24-hour garbage-collection time, single retry, and refetch-on-focus disabled.
- Zustand — lightweight client/UI state: useMatchesStore and useTeamsStore track the currently selected match/team; useUIStore tracks mobile menu open/closed state.

5.5 Data Layer (Services + Hooks)

services/worldcup.ts is the central API client: it wraps axiosClient calls to every backend endpoint and applies a client-side cache (lib/cache.ts) on top of the backend's own caching. Domain-specific service files (matchService, teamService, standingsService, playerService, bracketService, matchDetailsService) provide thin, typed wrappers around worldcupApi for use inside custom hooks (use-matches.ts, use-live-matches.ts, use-team.ts, use-standings.ts, use-top-scorers.ts, use-person.ts, useBracket.ts, use-matchDetails.ts, use-worldcup-news.ts).

5.6 Component Highlights

- PlayerProfileCard — two-column cinematic layout with hero photo, glassmorphism panels, and shirt-number watermark.
- TeamDetailsPage / TeamProfileBanner — Framer Motion–orchestrated animations, waving flag/cloth texture effect, amber/blue dual-accent styling.

- TopScoresCard (TopScoresCard.tsx) — rank-based dynamic styling (gold/silver/bronze treatment for top ranks), glassmorphism, Bebas Neue typography, lucide-react iconography.
- Bracket suite (Bracket.tsx, BracketColumn, Connector, MatchCard, FinalCard) — renders the knockout stage as a connected tree with skeleton loading state.
- LiveMatches dashboard (OverView.tsx/* components) — tab navigation with live status indicators, mini group overview, and news snippets.
- WorldCupBanner/CountDown — parallax hero banner with a tournament countdown.

5.7 Environment Setup

```
cd frontend
pnpm install
cp .env.example .env.local     # then set NEXT_PUBLIC_API_BASE_URL
pnpm dev        # http://localhost:3000
pnpm build
pnpm start
pnpm lint
pnpm type-check
```

6. Design System

Score90X follows a consistent cinematic, dark sports-broadcast aesthetic across every screen, matching the visual identity established across the project's component work.

6.1 Color Palette

Token	Hex	Usage
Background (base)	#081226	Primary dark navy background across all pages
Accent — Gold/Amber	Amber/Gold tones	Primary accent: ranks, highlights, CTAs, borders
Accent — Blue	Blue tones	Secondary accent: links, live indicators, hover states
Surface	Glassmorphism (backdrop-blur)	Card surfaces layered above the dark background

6.2 Typography

- Display/Headline: Bebas Neue — used for scores, ranks, and hero headings across cards.
- Body/UI: Poppins and Inter — loaded globally in the root layout for body copy and UI text.

6.3 Interaction Language

- Glassmorphism (backdrop-blur) panels layered over the dark base background.
- Hover interactions with glow/amber-accent effects on cards and buttons.
- Framer Motion–driven entrance and orchestration animations on detail pages (e.g. Team Details).
- Rank-based dynamic styling — e.g. gold/silver/bronze emphasis on top-scorer cards.

7. Core Data Types

Shared TypeScript types (frontend/src/types) define the domain model consumed throughout the UI:

7.1 Match

```
interface TeamSummary { id, name, crest, tla }
interface ScoreSummary {
  winner: 'HOME_TEAM' | 'AWAY_TEAM' | 'DRAW' | null;
  duration: string;
  fullTime: { home: number | null; away: number | null };
  halfTime: { home: number | null; away: number | null };
}
interface Match {
  id, status, utcDate, stage, group, matchday, lastUpdated,
  homeTeam: TeamSummary, awayTeam: TeamSummary, score
}
```

7.2 Other Core Types

- Team — squad, crest, colors, coach, and profile metadata.
- StandingGroup — group-stage standings table entries.
- TopScorer — player ranking, goals, assists, team association.
- Person — player/coach profile data.
- BracketMatch / MatchDetails — extended shapes used by the bracket and match-detail views.

8. Known Limitations & Roadmap

Area	Current State	Suggested Next Step
Database	Mongoose listed as a dependency; database/mongodb.ts is an empty scaffold	Wire up MongoDB for persistence (e.g. caching layer, favorites, user accounts)
Scheduled refresh	node-cron installed but unused	Add a cron job to pre-warm the cache ahead of TTL expiry for live matches
Auth	No authentication layer present	Add auth if user accounts/favorites are introduced
API docs	No OpenAPI/Swagger spec	Generate an OpenAPI spec from the existing route/controller layer
Testing	No automated test suite observed	Add unit tests for mapper/service logic and component tests for the UI